



Make optional<T&> trivially copyable (NB comment US 134-215)

Document number:

[P3836R2](#)

Date:

2025-11-05

Audience:

LEWG, LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Nevin Liber <nliber@anl.gov>

Steve Downey <sdowney@gmail.com>

GitHub Issue:

wg21.link/P3836/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/trivially-copyable-optional-ref.cow



ABSTRACT: Possibly due to a wording oversight, optional<T&> is not guaranteed to be trivially copyable. We propose to fix this in C++26. Due to certain design questions that concern LEWG, this issue is not simply handled as an LWG issue.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R1
 - 1.2 Changes since R0
- 2 **Introduction**
 - 2.1 Wording problem
- 3 **Motivation**



4	Design
4.1	Option 2 (no longer proposed)
5	Wording
6	Acknowledgements
7	References

§ 1. Revision history

§ 1.1. Changes since R1

- used more conventional "NB comment US 134-215" in title
- applied LEWG direction (*only* specify optional<T&> to be trivially copyable) after Kona 2025 poll

§ 1.2. Changes since R0

- added Steve Downey as co-author
- referenced related US NB comment 134
- rewrote §5. Wording

§ 2. Introduction

Making operations (at least construction) of optional<T&> trivial is the design intent of [P2988R12]:

”

3.3 Trivial construction

Construction of optional<T&> should be trivial, because it is straightforward to implement, and optional<T> is trivial. Boost is not.

However, possibly due to a wording oversight, trivial construction is not actually guaranteed, nor is trivial copyability in general. This status quo is inconsistent with the design of optional<T> as a whole: the special members (copy constructor, destructor, etc.) of the primary template are all specified to be trivial if the relevant operations on T are trivial. Therefore, optional<int*> is required to be trivially copyable, unlike optional<int&>.

This issue should be fixed in C++26. Our paper corresponds to the US national body comment 134.

§ 2.1. Wording problem



Intuitively, `optional<T&>` (see [P2988R12]) is a `T*`, wrapped optional. In fact, the synopsis as specified in ([optional.optional.ref]) is as follows:

```
template<class T>
class optional<T&> {
    [...]

    public:
        // [optional.ref.ctor], constructors
        constexpr optional() noexcept = default;

        [...]

        constexpr ~optional() = default;

        [...]

    private:
        T* val = nullptr; // exposition only
};
```

If the only non-static data member was `val`, `optional<T&>` would be trivially copyable according to [class.prop] definition of "class, trivially copyable". However, the implementation has the freedom to add extra non-static data members, bit-fields, and base classes, so no trivial operation is strictly guaranteed.

§ 3. Motivation

`optional<T&>` should be guaranteed to be trivially copyable for the following reasons:

- It makes the design consistent with the primary template `optional<T>`.
- Trivial copyability is a useful property in this case. It allows storing `optional<T&>` in a `struct` whose bytes are copied between processes, uploaded from CPU to GPU (e.g. in CUDA), etc.
- Trivial copyability also makes `optional<T&>` an implicit-lifetime class type, which makes it possible to start its lifetime in a `union` more easily via assignment operator, among other benefits.

§ 4. Design

There are two plausible options to fix the issue of `optional<T&>`:

1. Simply specify `optional<T&>` to be trivially copyable.
2. Additionally constrain its layout so it *actually* becomes a wrapper for a `T*`.

Both directions were considered during Kona 2025, but the room only had appetite for the first option, so only the first option was polled:



”

POLL: Apply the resolution described in NB comment “US 134-215 22.5.4.1 [optional.optional.ref.general] optional<T&> is a trivially copyable type” with option 1 (specify optional<T&> to be trivially copyable) from P3836R1 and send resolution to LWG.

SF	F	N	A	SA
12	2	1	1	0

Outcome: Consensus in favor

§ 4.1. Option 2 (no longer proposed)

R0 and R1 of this paper argued that option 2 would have provided more value to the user because trivial copyability is much more useful when the layout of a type is somewhat constrained. This makes it possible to `bit_cast<T*>(optional_ref)` or to encode `optional<T&>` using `memcpy` with *some* guarantees regarding its layout.



EXAMPLE: If `optional<int&>` is trivially copyable *and* its layout is that of a pointer, we can do the following:

```
int* legacy_api(); // returns nullable pointer

optional<int&> modern_api() {
int* p = legacy_api();
return p ? *p : optional<int&>{};
    return bit_cast<optional<int&>>(legacy_api());
}
```

There are no "null references", so bit-casting is the easiest way to wrap old pointer-based APIs like this.



NOTE: The layout assumptions could still be false for an implementation where the layout of "wrapper `structs`" and the types they wrap is different, but that concern is largely hypothetical.

However, LEWG found option 2 to be unconvincing because:

- That de-facto leaks the private `T*` data member in an `optional<T&>`; why not just make it public?
- Specifying the layout of `optional<T&>` was arguably not within the scope of NB comment US 134-215.
- The change lacked motivation and lacked symmetry with the rest of the standard library.

§ 5. Wording



The wording changes are relative to [N5014].

Immediately following [optional.optional.ref.general] paragraph 1, append a paragraph as follows:



Each type `optional<T&>` is a trivially copyable class ([class.prop]).

§ 6. Acknowledgements

Thanks to Brian Bi for reviewing the flawed wording in R0 of this paper.

§ 7. References

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P2988R12] *Steve Downey, Peter Sommerlad*. `std::optional<T&>` (2025-04-04)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf>